Universidad de La Sabana

Pruebas Unitarias y TDD

El motor de la calidad en la cultura DevOps

Unidad 3 | Maestría en Profundización en Ingeniería de Software

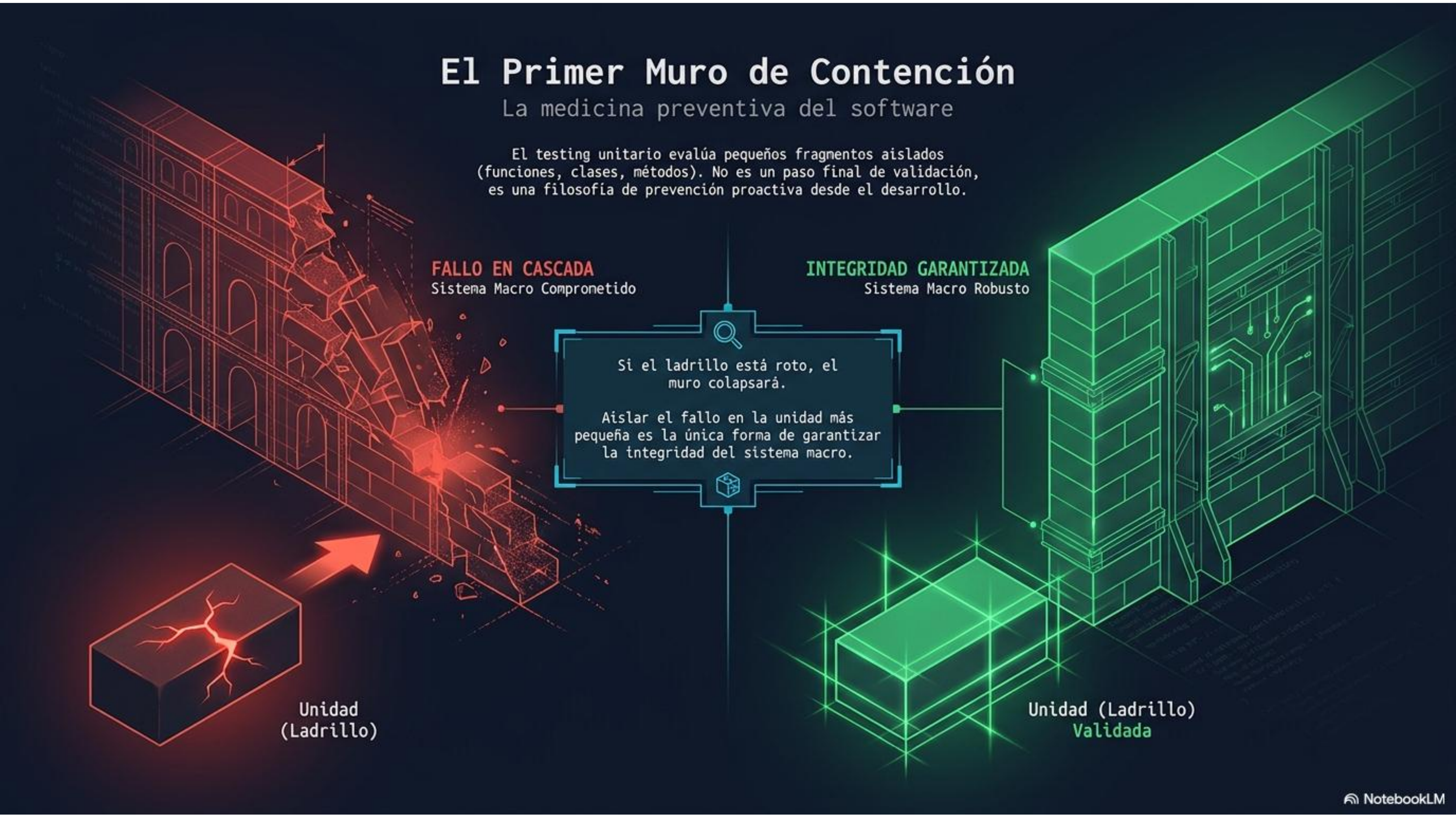
Autor: César Augusto Vega Fernández

NotebookLM

El Primer Muro de Contención

La medicina preventiva del software

El testing unitario evalúa pequeños fragmentos aislados (funciones, clases, métodos). No es un paso final de validación, es una filosofía de prevención proactiva desde el desarrollo.



NotebookLM

El Ecosistema de la Calidad

Objetivos estratégicos de la Unidad 3



1. Automatizar (JUnit)

Implementar el estándar de automatización para validaciones precisas y deterministas.



2. Simular (Mockito)

Aislar componentes simulando dependencias para controlar variables externas.



3. Diseñar (TDD)

Adoptar el desarrollo dirigido por pruebas como cultura central, no solo como técnica.



4. Medir (Code Coverage)

Cuantificar la salud técnica usando métricas como compuertas objetivas de calidad.

NotebookLM

Anatomía de una Prueba Determinista

Estandarización estructural: AAA vs BDD



Sin importar el lenguaje de programación, este es el ADN inmutable de una prueba legible y mantenible.

NotebookLM

El Arte de la Simulación y el Aislamiento

Mockito y el control total de las dependencias

Una prueba unitaria que interactúa con una base de datos real o una red NO es unitaria; es una prueba de integración. Necesitamos cuarentena total.

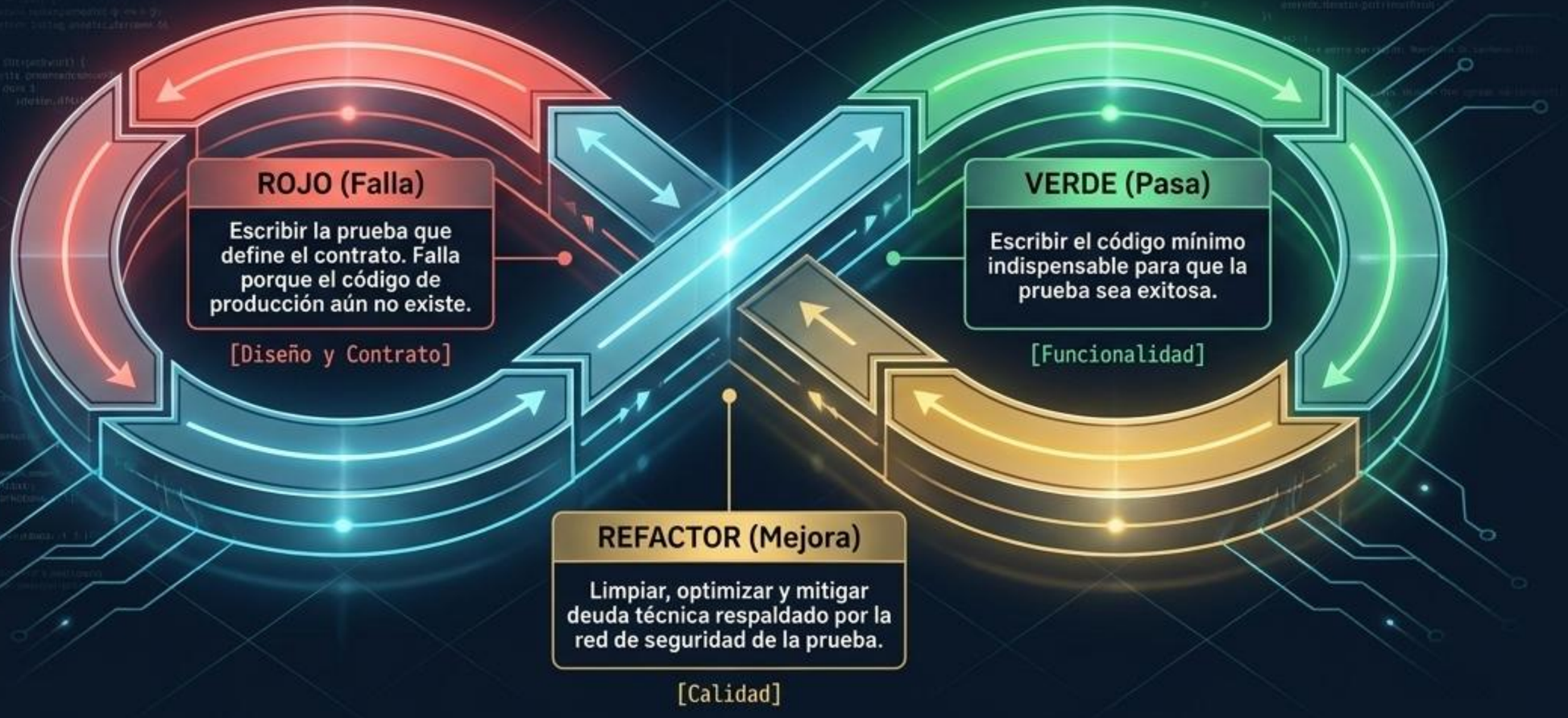


Mockito nos otorga **control total** sobre el tiempo y el espacio de nuestros datos, eliminando la inestabilidad de la infraestructura.

NotebookLM

El Ritmo del Desarrollo Ágil: TDD

Test-Driven Development como motor de diseño



TDD no es una técnica de testing; es una técnica de diseño de software impulsada por validaciones automatizadas.

NotebookLM

Métricas de Salud Técnica

El Code Coverage como “Quality Gate”

En la cultura DevOps, las percepciones subjetivas se transforman en datos objetivos. La cobertura no es solo estadística, es una compuerta.



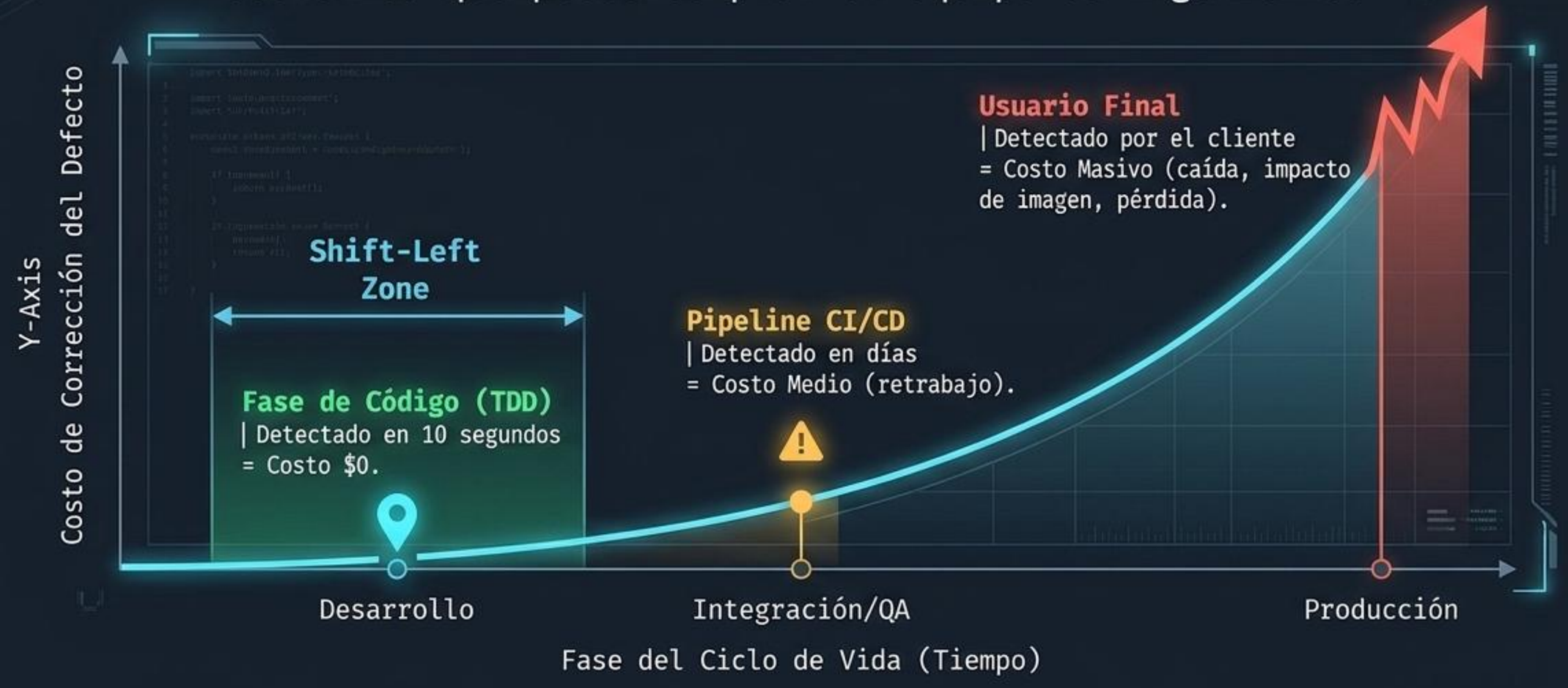
Requisito crítico para el Proyecto Integrador del curso:
Lograr >75% de cobertura con trazabilidad demostrable.

NotebookLM

Caso Especial: El Error de los 10 Segundos

La economía del Shift-Left Testing

"Las pruebas unitarias son la póliza de seguro financiero más económica que puede comprar un equipo de ingeniería."



NotebookLM

Autopsia de una Mala Prueba

Errores comunes que destruyen la confianza del equipo



1. Pruebas Tautológicas

Escribir aserciones sin sentido. Generan una falsa cobertura técnica sin validar la verdadera lógica de negocio.

2. Falsa Seguridad (Sin Aserciones)

Ejecutar la invocación pero omitir la aserción. La prueba "pasa" solo porque el código no colapsa, creando falsos positivos.

3. Dependencia de Datos Volátiles

Probar usando fechas del reloj del sistema o conectándose a una BD productiva en vivo. Causa pruebas intermitentes ("Flaky").

NotebookLM

Ciudadanos de Primera Clase

Buenas prácticas para suites de pruebas sostenibles

"El código de prueba no es un subproducto desechable. Debe ser tratado con el mismo rigor arquitectónico que el núcleo del sistema."



NotebookLM

El ROI de la Confianza

Escribir código rápido vs. Entregar valor rápido

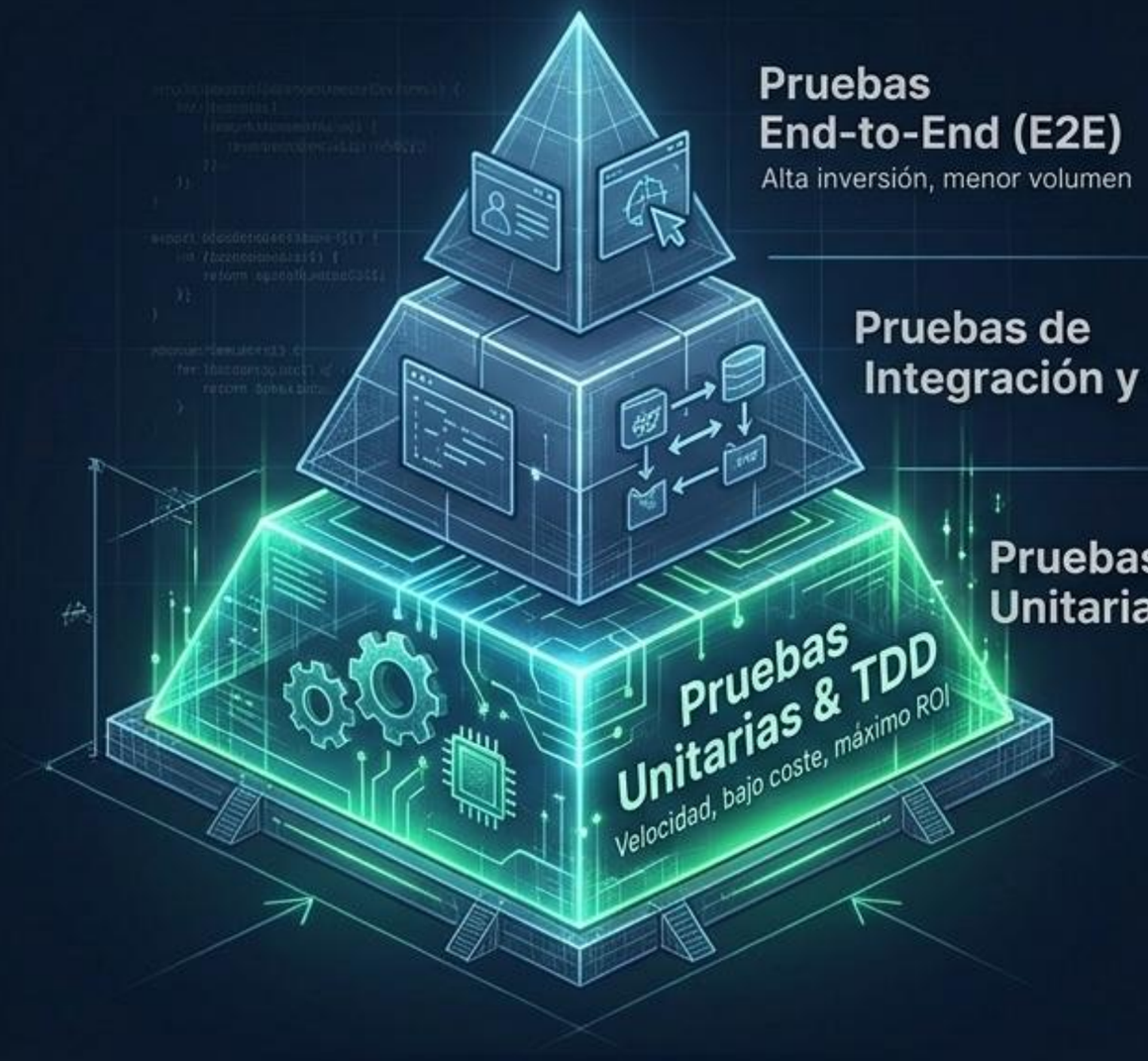


En el desarrollo moderno, el testing unitario no "quita" tiempo; regala confianza. La verdadera agilidad no consiste en codificar apresuradamente, sino en no tener que regresar a arreglar lo que ya estaba hecho.

NotebookLM

Conclusiones de la Unidad

Los cimientos inquebrantables de la automatización



Herramientas Esenciales

Frameworks como JUnit y Mockito son herramientas indispensables para las decisiones de un Arquitecto de Software moderno.

Fundamento DevOps

El éxito de CI/CD en la cultura DevOps depende matemáticamente de una base inmensa de pruebas deterministas.

Su Misión

Su misión: Aplicar esta solidez estructural en la codificación y despliegue de su Proyecto Integrador.

NotebookLM